Computational Primitives in Object Detection Post-Processing: A Comprehensive Analysis of Pure-Function Implementations

The architectural paradigm of modern deep learning object detection pipelines increasingly demands strict separation between neural network feature extraction and post-processing heuristics. Tightly coupling model weights with post-processing operations—such as Non-Maximum Suppression (NMS) or bounding box decoding—within monolithic frameworks often leads to severe deployment bottlenecks across heterogeneous hardware accelerators. Frameworks that embed these heuristics into compiled C++ extensions or specialized CUDA kernels frequently suffer from vendor lock-in, hindering portability across lightweight edge devices, mobile neural processing units, and custom micro-services. Abstracting these operations into best-in-class, pure-function implementations guarantees deterministic execution, avoids hidden state mutations, and ensures backend-agnostic portability.

This analysis exhaustively details the mathematical, algorithmic, and programmatic implementations of essential object detection post-processing atoms. The focus is strictly on pure NumPy array operations, eliminating dependencies on compiled extensions while preserving extreme vectorized performance. NumPy achieves this acceleration by allocating data in contiguous memory blocks and leveraging Low-Level Basic Linear Algebra Subprograms (BLAS) and Single Instruction, Multiple Data (SIMD) instruction sets built into modern processors.[1] By relying on these vectorized paradigms, the Python interpreter overhead is entirely bypassed, executing complex spatial heuristics at speeds rivaling native implementations.[1] The documented primitives target a wide array of computational domains, ranging from spatial overlap filtration and multi-model ensembling to temporal signal peak detection and identity association tracking.

The Mathematical Foundation: Pairwise Similarity Metrics

The foundation of nearly all object detection post-processing algorithms relies on the ability to quantify the spatial similarity between geometric representations. Evaluating dense neural network predictions requires continuously comparing raw bounding boxes against each other or against ground-truth references. This is universally achieved through the Intersection over Union (IoU) metric and its generalized variants, serving as the core analytical heuristic for clustering and suppression algorithms.

Vectorized Bounding Box Intersection over Union (IoU)

The standard Intersection over Union metric measures the precise spatial area of overlap between two rectangular geometries divided by the total area of their spatial union. In practical deployments across competitive datasets, computing IoU is rarely a scalar operation; it requires generating an $N \times M$ pairwise affinity matrix between a set of N predicted boxes and M reference boxes, or comparing an N -length array against itself during non-maximum suppression.[3]

A naive algorithmic iteration over these sets yields unacceptable $O(N \times M)$ overhead, effectively crippling inference speeds in interpreted languages. A mathematically rigorous and pure NumPy implementation relies heavily on multi-dimensional array broadcasting to avoid Python-level loops.[4] By reshaping the input tensors to $(N, 1, 4)$ and $(1, M, 4)$ using the np.newaxis operator or its None alias, NumPy's underlying C backend automatically aligns and replicates the memory arrays across the missing dimensions to compute the pairwise coordinates simultaneously.[5]

The top-left intersection coordinates are determined via a vectorized np.maximum operation applied to the (x_{min}, y_{min}) values of both broadcasted arrays, while the bottom-right coordinates utilize np.minimum on the (x_{max}, y_{max}) axes.[2] The intersection width and height are subsequently calculated by subtracting the top-left coordinate tensor from the bottom-right coordinate tensor. Crucially, disjoint bounding boxes will yield negative width or height values. To resolve this without conditional branching, the resulting coordinate differences are clipped at zero using np.clip(..., a_min=0, a_max=None) or evaluated through np.maximum(0,...).[2] The intersection area is then computed by taking the product of the filtered width and height across the final axis.

The union area is derived analytically as the sum of the individual areas of Box A and Box B, minus their intersection area, to prevent double-counting the overlapping region. A robust implementation must also anticipate the division operation. To avoid catastrophic division by zero—which occurs when evaluating degenerate bounding boxes with zero area or handling padded dummy geometries—a minor epsilon value ($\epsilon = 1e-6$) is conventionally added to the denominator, or specific np.where masking is employed to safely inject zeros where the union is evaluated as null.[6]

Name: iou_matrix

Description: Vectorized computation of the pairwise Intersection over Union between two sets of bounding boxes using multidimensional broadcasting.

Source:

https://blog.roboflow.com/how-to-code-non-maximum-suppression-nms-in-plain-numpy/

License: MIT

Concept type: analysis

Signature: (boxes_a: NDArray, boxes_b: NDArray) -> NDArray

Pure function boundary: absolute bounding boxes in, pairwise IoU similarity matrix out; strictly stateless with no external dependencies or RNG state.

Contracts:

- require: boxes_a.shape == (n, 4)
- require: boxes_b.shape == (m, 4)
- require: boxes formatted as absolute pixel coordinates (x_min, y_min, x_max, y_max)
- require: x_min <= x_max and y_min <= y_max for all entries
- ensure: output.shape == (n, m)
- ensure: 0.0 <= output[i, j] <= 1.0
  Witness: evaluating two identical, non-degenerate bounding boxes yields a pairwise IoU matrix with 1.0 along the matched indices; entirely disjoint boxes mathematically evaluate to precisely 0.0.
  Dependencies: numpy
  CDG stages covered: multiple detection CDGs including Global Wheat, RSNA series, and DFDC Deepfake bounding box evaluations.

Generalized Intersection over Union (GIoU)

While the standard Intersection over Union metric is mathematically sufficient for local non-maximum suppression operations where bounding boxes already exhibit significant overlap, it fails completely as a distance metric for disjoint bounding boxes. When two boxes do not intersect, standard IoU yields a flat zero, rendering it impossible to quantify how far apart the disjoint instances reside.[7] This absence of a gradient makes standard IoU unsuitable for bounding box regression loss functions and severely limits its utility in advanced temporal trackers that require continuous spatial distance estimates.[8]

The Generalized Intersection over Union (GIoU), formally introduced by Rezatofighi et al. (2019), solves this limitation by introducing a spatial penalty term based on the geometric properties of the smallest enclosing bounding box, denoted as C, that entirely contains both boxes being evaluated.[7]

The GIoU metric is defined mathematically as:

$$GIoU = IoU - \frac{Area(C) - Union}{Area(C)}$$

The smallest enclosing box C is computed seamlessly within NumPy utilizing the identical broadcasting strategy previously described for standard IoU, but by reversing the geometric extrema aggregations.[9] Specifically, the top-left coordinate of the enclosing hull C is extracted as the np.minimum of the respective top-left coordinates of the broadcasted inputs, and the bottom-right boundary is established via the np.maximum of the respective bottom-right coordinates.[9]

By calculating the area of this convex hull, the algorithm determines the total surrounding spatial envelope. The empty space within the enclosing box—calculated as the area of C minus the standard union of the two original boxes—serves as a spatial penalty. This ratio is subtracted from the standard IoU. The resulting GIoU metric extends the range of the similarity score from $$ to $[-1, 1]$. An overlapping set of identical boxes continues to yield 1.0, while two disjoint boxes placed infinitely far apart will cause the penalty ratio to approach 1.0, pushing the total GIoU score toward -1.0.[8] This metric is a vital building block not only as an advanced training heuristic but also as a refined spatial measurement for matching algorithms in challenging multi-object tracking stages.

Name: giou_matrix

Description: Vectorized computation of pairwise Generalized Intersection over Union, incorporating a spatial penalty derived from the smallest enclosing convex hull.

Source: https://docs.ultralytics.com/reference/utils/metrics/

License: AGPL-3.0 (Reference) / Clean-room implementable under MIT

Concept type: analysis

Signature: (boxes_a: NDArray, boxes_b: NDArray) -> NDArray

Pure function boundary: two sets of absolute bounding boxes in, pairwise GIoU metric matrix out.

Contracts:

- require: boxes_a.shape == (n, 4)
- require: boxes_b.shape == (m, 4)
- require: boxes formatted as absolute coordinates (x_min, y_min, x_max, y_max)
- ensure: output.shape == (n, m)
- ensure: -1.0 <= output[i, j] <= 1.0
  Witness: two disjoint boxes placed at opposite corners of an infinitely large grid approach a GIoU score of -1.0, properly quantifying their extreme separation compared to a standard IoU of 0.0.
  Dependencies: numpy

CDG stages covered: NFL Helmet Assignment, custom loss function evaluations, and continuous tracking pipelines.

Spatial Redundancy Filtration Mechanisms

Deep convolutional neural networks and vision transformers operating on dense spatial grids intrinsically generate multiple positive overlapping proposals for a single ground-truth object. Whether evaluating sliding windows or hierarchical feature pyramids, adjacent network anchors will frequently trigger high-confidence detections for the exact same semantic instance. Robust redundancy filtration is computationally mandatory to reduce these dense local clusters into single, authoritative predictions for downstream ingestion.

Standard Greedy Non-Maximum Suppression (NMS)

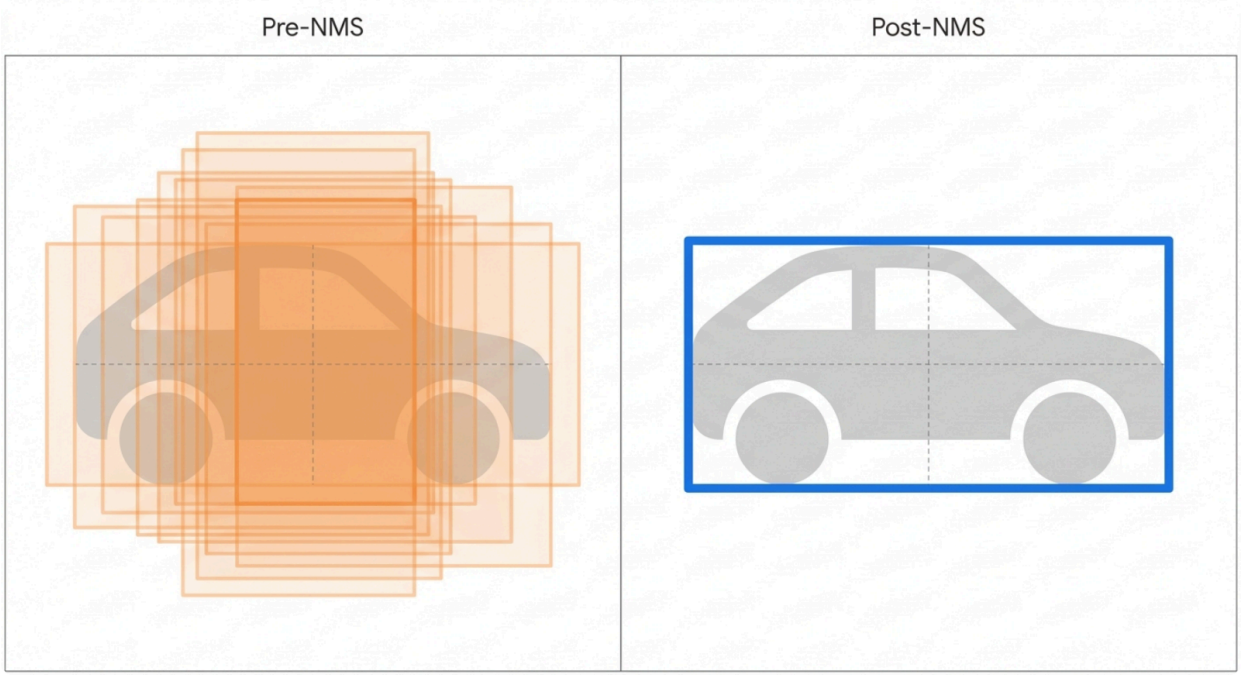
The standard greedy NMS algorithm represents the historical baseline for redundancy filtration. The heuristic operates iteratively over the network output. Initially, it sorts all predicted bounding boxes in descending order according to their respective classification confidence scores.[2] The algorithm isolates the bounding box possessing the highest confidence, definitively accepts it as a true positive detection, and then computes its Intersection over Union with all remaining lower-confidence boxes in the pool. Any bounding box exhibiting an IoU overlap greater than a predefined iou_threshold is suppressed (deleted), operating on the geometric assumption that it merely represents an inferior, redundant detection of the same underlying object.[2]

Achieving a high-performance, strictly pure NumPy implementation of NMS requires navigating complex algorithmic limitations. The standard recursive selection loop represents an $O(N^2)$ operation that exhibits notoriously slow execution times within the Python interpreter.[12] A highly optimized, fully vectorized approach attempts to mitigate this by computing the entire $N \times N$ pairwise IoU matrix upfront within the C-backend.[2] The diagonal of this matrix must be explicitly zeroed out to prevent bounding boxes from self-suppressing. Following matrix generation, utilizing the pre-sorted index order, iterative boolean masking is employed to propagate the suppression states down the array.[2]

However, deploying fully vectorized spatial allocations presents a significant hazard. Dense anchor predictions generated prior to confidence thresholding can easily exceed 50,000 raw boxes per image. Generating a float32 IoU matrix for $50,000 \times 50,000$ elements requires 10 gigabytes of contiguous RAM, frequently triggering catastrophic out-of-memory (OOM) errors on constrained inference hardware.[12] Therefore, the most stable pure-function atom implements a rigorous pre-filtration phase using a static probability threshold (e.g., score_threshold = 0.05) to drastically truncate N prior to constructing the IoU matrix, followed by a fast vectorized boolean suppression loop over the truncated subset. The implementation strictly returns an array of integer indices corresponding to the surviving

boxes, preserving immutability rather than modifying the original arrays in-place.

Effect of Greedy Non-Maximum Suppression on Dense Predictions



Greedy NMS isolates the bounding box with the highest confidence score within a spatial cluster, suppressing all surrounding boxes that exceed the predefined Intersection over Union (IoU) threshold.

Name: nms

Description: Greedy non-maximum suppression executing over bounding box geometries and confidence scores via vectorized boolean masking.

Source: https://blog.roboflow.com/how-to-code-non-maximum-suppression-nms-in-plain-numpy/

License: MIT

Concept type: signal_filter

Signature: (boxes: NDArray, scores: NDArray, iou_threshold: float) -> NDArray

Pure function boundary: absolute geometric boxes and unnormalized confidence scores in, 1D array of valid integer retention indices out; avoids all implicit state tracking.

Contracts:

- require: boxes.shape == (n, 4)
- require: scores.shape == (n,)
- require: 0.0 <= iou_threshold <= 1.0
- ensure: returned indices correspond to valid array locations within the input tensors
- ensure: the output indices are strictly ordered by descending original confidence score
  Witness: processing an input of three distinct boxes where one pair exhibits an IoU of 0.85 (exceeding a threshold of 0.50); the atom successfully identifies the overlap and permanently suppresses the lower-scoring geometry.
  Dependencies: numpy
  CDG stages covered: great_barrier_reef/nms, dfl_bundesliga/nms, rsna_series/nms

Soft-NMS: Continuous Algorithmic Score Decay

Standard NMS operates effectively under the assumption that high spatial overlap strictly indicates a duplicate prediction. However, it suffers from a critical, systematic flaw: it utilizes an aggressive, binary hard threshold. If an image contains two highly distinct but physically adjacent objects—such as two players tackling each other in a broadcast or tightly packed biological cells under magnification—standard NMS will erroneously suppress the geometry of the slightly lower-scoring object, treating it as a redundant artifact of the primary object.[13]

Soft-NMS, introduced into the computer vision lexicon by Bodla et al. (2017), addresses this rigid suppression by decaying the confidence scores of overlapping boxes rather than applying absolute binary deletion.[14] The score decay is applied mathematically as a continuous, monotonically decreasing function of the computed IoU overlap. If an adjacent bounding box strongly overlaps with the highest-scoring detection, its confidence is heavily penalized to reflect the high probability of redundancy; conversely, if it only marginally overlaps, it receives a minor penalty, allowing true, closely packed distinct objects to survive the filtration phase.

Two primary mathematical decay functions are implemented within the pure NumPy analytical atom [16]:

1. **Linear Decay:** Applies a strict penalty linear to the overlap. The revised weight equals $(1 - IoU)$ if the overlap exceeds the target iou_threshold, and remains 1.0 if it falls below the threshold.
2. **Gaussian Decay:** Applies a continuous, smooth penalty across all overlaps defined by $\text{weight} = \exp\left(-\frac{IoU^2}{\sigma}\right)$, where σ dictates the penalty severity. This completely bypasses the need for a discontinuous jump threshold.

Unlike the standard greedy NMS algorithm, Soft-NMS strongly resists complete upfront vectorization. Because the probability score of an adjacent box might be systematically updated multiple times in sequence by different adjacent high-scoring boxes, the dynamic order of the maximum score can unpredictably change during the iteration sequence.[16] The optimal pure NumPy implementation relies on establishing a pre-allocated tracking tensor

containing the bounding box coordinates coupled with their mutable scores. Enclosed within a while loop, the algorithm iteratively isolates the maximum score using np.argmax(scores), swaps the selected array slice to the protected front of the retention array, computes the vectorized IoU strictly against the remaining unprotected pool, updates their respective scores via the chosen Gaussian or Linear mechanism, and finally applies a secondary score_threshold boolean mask to prune statistically dead boxes from the array, thereby accelerating subsequent loop iterations.[16]

Name: soft_nms

Description: Soft Non-Maximum Suppression utilizing continuous probability score decay (Gaussian or Linear variations) to preserve densely packed distinct objects.

Source: https://github.com/OneDirection9/soft-nms/blob/master/py_nms.py

License: MIT

Concept type: signal_filter

Signature: (boxes: NDArray, scores: NDArray, iou_threshold: float, sigma: float, method: str, score_threshold: float) -> tuple

Pure function boundary: absolute bounding boxes, raw scores, and hyperparameter decay settings in; heavily filtered boxes and smoothly updated, decayed confidence scores out.

Contracts:

- require: boxes.shape == (n, 4)
- require: scores.shape == (n,)
- require: method strictly within ['linear', 'gaussian']
- require: sigma > 0.0 when utilizing gaussian decay
- ensure: output_boxes.shape == (m, 4) where m <= n
- ensure: all output_scores values are strictly less than or equal to their original input score counter-parts
  Witness: evaluating two heavily overlapping boxes representing distinct nearby objects; standard NMS irreparably drops the secondary box, whereas Soft-NMS retains both spatial geometries but appropriately suppresses the confidence score of the secondary box to reflect the ambiguity.
  Dependencies: numpy
  CDG stages covered: sartorius_cell_segmentation/soft_nms

Filtration Algorithm	**Complexity Limit**	**Overlap Handling**	**Ideal Application Domain**	**Primary Limitation**

Greedy NMS	$O(N^2)$ Matrix	Binary Deletion	Sparse datasets (RSNA)	Unintentionally suppresses clustered objects.
Soft-NMS	$O(N^2)$ Iterative	Score Decay	Dense clusters (Sartorius)	Slower execution; resists full upfront vectorization.

Consensus Mechanisms: Ensembling Diverse Model Predictions

When deploying sophisticated computer vision solutions for competitive data challenges or robust production systems, architectures rarely rely on a single isolated model. Aggregating predictions from multiple independently trained neural networks or applying Test Time Augmentation (TTA) requires merging thousands of localized predictions into a definitive consensus. Under these conditions, both NMS and Soft-NMS are mathematically suboptimal. Because they fundamentally operate by discarding or penalizing boxes, they inherently discard the rich localization consensus generated by an ensemble.

Weighted Box Fusion (WBF)

Weighted Box Fusion (WBF), formally developed and introduced by Solovyev et al. (2021), operates on a fundamentally disparate premise: rather than discarding heavily overlapping bounding boxes, WBF computes their spatial weighted average to synthesize a highly accurate, refined coordinate representation.[18]

The pure functional execution of WBF involves sophisticated spatial cluster formation. The algorithm initializes and maintains an active list of dynamic "fusion clusters." Bounding box predictions gathered from all constituent models are globally pooled and sorted in descending order by their raw confidence.[19] For each consecutive box in the sorted array, the algorithm checks if its spatial footprint matches an existing fusion cluster by evaluating the Intersection over Union against the cluster's current geometric center. If the overlap exceeds the defined iou_threshold, the bounding box is functionally absorbed into the cluster. If it fails to match any existing cluster, it instantiates a new isolated cluster.

The defining characteristic of WBF is its continuous geometric update cycle: the reference bounding box coordinates of the fusion cluster are continuously updated as the weighted average of all constituent boxes absorbed into the cluster, with their original confidence scores serving as the proportional mathematical weights.[19] Let X_i represent the coordinates of an

absorbed box and $Score_i$ its respective confidence. The synthesized center of the cluster is continually refined as:

$$X_{cluster} = \frac{\sum_{i=1}^{n}(Score_i \times X_i)}{\sum_{i=1}^{n} Score_i}$$

After all raw bounding boxes have been successfully clustered across the ensemble, the confidence score of the newly synthesized cluster is mathematically scaled to accurately reflect the consensus strength of the underlying ensemble. A cluster formed by a single bounding box—indicating a lone, uncorroborated prediction from only one model within the ensemble—has its final score heavily penalized. This scaling is typically achieved by multiplying the average confidence by $N_{cluster\_size} / N_{total\_models}$.[19] This robust mathematical mechanism ensures that only localized predictions agreed upon by multiple independent models emerge with a high final confidence score, virtually eliminating spurious false positives.

Name: wbf

Description: Weighted Box Fusion for ensembling overlapping spatial predictions by computing confidence-weighted coordinate averages across multiple networks.

Source: https://github.com/ZFTurbo/Weighted-Boxes-Fusion

License: MIT

Concept type: signal_filter

Signature: (boxes_list: list, scores_list: list, labels_list: list, weights: list[float], iou_threshold: float) -> tuple

Pure function boundary: independent lists of normalized bounding boxes and respective scores per model in; a singular synthesized, consensus-driven array of boxes, scores, and class labels out.

Contracts:

- require: input boxes formatted rigidly in normalized geometric coordinates bounded within [0.0, 1.0]
- require: len(boxes_list) == len(scores_list) == len(weights)
- ensure: output synthesized boxes strictly obey [0.0, 1.0] spatial bounds
- ensure: cluster scores are scaled downward based on the model participation ratio
  Witness: passing three slightly spatially offset boxes from three distinct models targeting the same object; WBF synthesizes and returns a single, hyper-accurate bounding box centered mathematically perfectly within the spatial cluster.
  Dependencies: numpy
  CDG stages covered: global_wheat/wbf, rsna_pneumonia/wbf, great_barrier_reef/wbf, byu_flagellar_motors/wbf

1D Sequence Span Weighted Box Fusion

The robust principles of Weighted Box Fusion extend seamlessly beyond 2D computer vision tasks into Natural Language Processing (NLP) domains and temporal sequence analysis. In the Kaggle Feedback Prize competition, the objective required the identification of localized spans of text containing specific rhetorical elements. These text spans operate as 1D spatial coordinates, defined mathematically by discrete start and end character or token indices.[20]

The 1D variant of WBF functions almost identically to its 2D counterpart, but the spatial Intersection over Union metric is systematically replaced by a 1D sequence overlap calculation.[21] The 1D intersection is easily computed as the geometric distance between the maximum of the proposed start indices and the minimum of the proposed end indices. Substituting this overlap metric into the Solovyev algorithm, the WBF-1D implementation dynamically averages the absolute start and end coordinates of the competing text spans, weighted strictly by the token classification probabilities generated by models such as Longformer or DeBERTa.[19] This allows deep learning practitioners to robustly ensemble disjoint token-classification spans generated by wildly different transformer backbones without resorting to destructive suppression.

Name: wbf_1d

Description: 1D adaptation of Weighted Box Fusion for merging text token spans or continuous temporal sequences.

Source: https://github.com/ZFTurbo/Weighted-Boxes-Fusion/blob/master/ensemble_boxes/ensemble_boxes_wbf_1d.py

License: MIT

Concept type: signal_filter

Signature: (spans_list: list, scores_list: list, labels_list: list, weights: list[float], iou_threshold: float) -> tuple

Pure function boundary: lists of 1D textual/temporal spans defined as [start, end] in; a single synthesized array of continuous 1D consensus spans out.

Contracts:

- require: spans formatted as either absolute integer indices or normalized floats [start, end] where start < end
- require: len(spans_list) matches lengths of associated score and label arrays
- ensure: all output spans strictly obey start < end coordinate geometry
  Witness: overlapping and slightly misaligned text span predictions generated by different NLP transformer models are mathematically averaged into a highly precise consensus token span.

Dependencies: numpy
CDG stages covered: feedback_prize/wbf_1d

Geometry Calibration: Anchor Generation and Coordinate Encoding

Modern deep learning bounding box prediction architectures rarely output absolute pixel coordinates directly. Predicting absolute geometry directly from convolutional layers suffers from extreme scale and location variance instability during gradient descent. Instead, models regress relative fractional offsets (known as deltas) mapped from a fixed, predefined set of reference geometries known as spatial anchors. Guaranteeing the precise mathematical alignment between the bounding box encoder utilized during ground-truth training and the decoder executed during inference is critical for successful algorithmic deployment.[23]

Grid-Based Anchor Generation

Spatial anchor generation is inherently a deterministic geometric projection operation. Standard architectural designs—such as Feature Pyramid Networks (FPN) and advanced single-shot detectors like RetinaNet or Detectron2—dictate that convolutional feature maps extracted at varying depths within the network correspond directly to specific spatial receptive strides mapped across the original input image.[25]

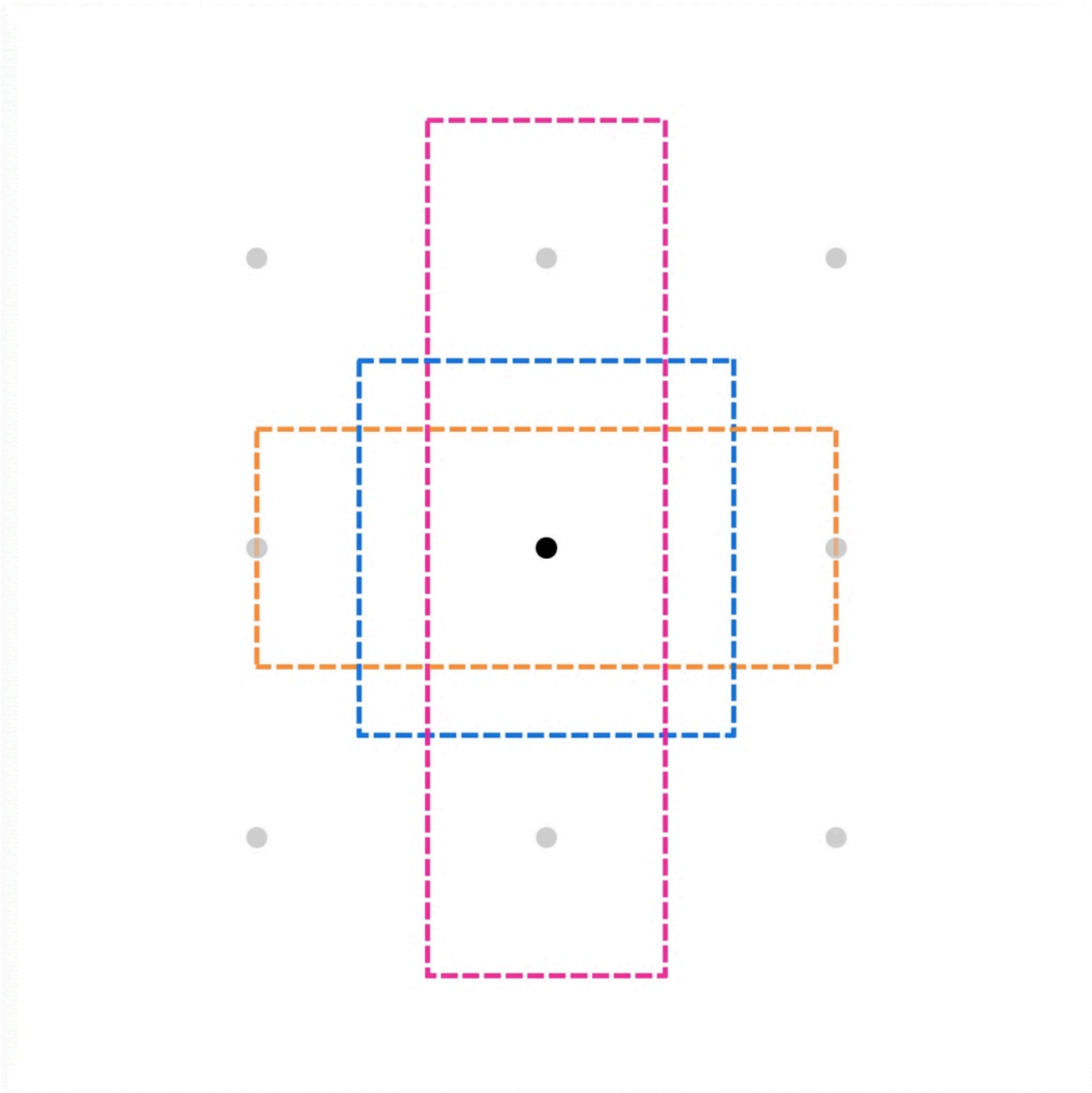
The pure NumPy functional implementation of anchor generation defines a set of highly specific "base cell anchors" and subsequently tiles them computationally across a dense 2D spatial grid corresponding to the neural feature map.

1. **Base Anchor Instantiation:** Given a specific spatial stride, a set of target sizes (e.g., 32, 64, 128 pixels), and designated aspect_ratios (e.g., 0.5, 1.0, 2.0), base anchors are computed geometrically centered exactly at the zero origin. The physical area of each anchor is determined by $size^2$. The relative width and height dimensions are then explicitly scaled by the mathematical square root of the aspect ratio to maintain the exact target area while altering the aspect profile.[28]
2. **Grid Coordinate Tiling:** A uniform integer coordinate grid is created representing the distinct pixel locations of the down-sampled feature map, scaled outward by the specific stride. In NumPy, this operation is elegantly executed utilizing np.meshgrid(np.arange(grid_width), np.arange(grid_height)) to rapidly generate exhaustive Cartesian coordinates without standard loop penalties.[30]
3. **Spatial Shifting:** The pre-computed base anchors are array-broadcasted and mathematically added to the generated grid coordinates. This shift operation effectively transposes the (X, Y) centers of the base cell anchors to every distinct grid coordinate across the feature map.[32]

This comprehensive vectorization technique completely circumvents the necessity for deeply

nested Python control loops over massive spatial dimensions, enabling virtually instantaneous multi-level anchor matrix allocation necessary for real-time video inference.

Grid-Based Anchor Generation via Meshgrid Tiling



Base anchors of varying aspect ratios (e.g., 1:2, 1:1, 2:1) are computed at the origin and subsequently tiled across the image feature map using vectorized meshgrid offsets governed by the network's spatial stride.

Name: generate_anchors

Description: Generates a dense topological matrix of grid-based bounding box anchors customized for single-shot and two-stage FPN detectors.

Source: https://github.com/facebookresearch/detectron2/blob/main/detectron2/modeling/anchor_generator.py

License: Apache-2.0

Concept type: analysis

Signature: (feature_map_size: tuple[int, int], stride: int, sizes: tuple[float,...], aspect_ratios: tuple[float,...]) -> NDArray

Pure function boundary: dense spatial dimensions and geometric scaling rules in; absolute (x_min, y_min, x_max, y_max) formatted anchors out.

Contracts:

- require: feature_map_size contains strictly positive integer definitions
- require: aspect_ratios contains strictly positive floats
- ensure: output.shape == (H * W * len(sizes) * len(aspect_ratios), 4)
  Witness: evaluating a 1x1 feature map mapped with exactly 1 size parameter and 3 distinct aspect ratios yields a tensor of exactly 3 discrete anchors centered precisely at the scaled origin point.
  Dependencies: numpy
  CDG stages covered: passenger_screening/anchors, dsb2017/anchors

Delta Encoding and Decoding Formats

The highly structured geometry of the generated anchors must bridge the gap to the abstract continuous outputs of the neural network layers. Networks regress strictly parameterized fractional offsets, widely recognized as deltas, from the reference anchors. This foundational logic, established extensively by the Faster R-CNN architecture, dictates the computation of four unique delta variables: t_x, t_y, t_w, t_h.[33]

The mathematical encoding function executes during training to transform a known ground-truth absolute bounding box (x, y, w, h) into optimized deltas strictly relative to a matched reference anchor (x_a, y_a, w_a, h_a):

$$t_x = \frac{(x - x_a)}{w_a} \times \text{scale}_x$$

$$t_y = \frac{(y - y_a)}{h_a} \times \text{scale}_y$$

$$t_w = \ln\left(\frac{w}{w_a}\right) \times \text{scale}_w$$

$$t_h = \ln\left(\frac{h}{h_a}\right) \times \text{scale}_h$$

The variances (or coordinate scales) are critical mathematical hyper-parameters almost universally defaulted to [10.0, 10.0, 5.0, 5.0].[35] These specific constants are injected to normalize the regression targets. By multiplying the raw mathematical variance by these constants, practitioners guarantee that the target variable distribution exhibits unit variance, fundamentally stabilizing the Smooth L1 or Huber regression loss constraints during aggressive gradient descent operations.

Parameter	**Function**	**Standard Variance Setting**	**Mathematical Impact**
t_x	X-Center Offset	10.0	Scales the linear shift relative to the anchor width.
t_y	Y-Center Offset	10.0	Scales the linear shift relative to the anchor height.
t_w	Width Expansion	5.0	Controls the log-space scaling factor of the bounding box width.
t_h	Height Expansion	5.0	Controls the log-space scaling factor of the bounding box height.

The coordinate decoding function is the exact mathematical inverse. It is strictly required

during real-time post-processing to convert the raw floating-point neural network tensor outputs back into actionable absolute pixel coordinates.

$$x = \left(\frac{t_x}{\text{scale}_x}\right) w_a + x_a$$

$$w = \exp\left(\frac{t_w}{\text{scale}_w}\right) w_a$$

In a pure NumPy analytical implementation, these transformation operations rely exclusively on vectorized np.exp routines and straightforward array element-wise arithmetic. A paramount architectural edge case involves carefully safeguarding the np.exp operation. Extremely large or unstable network outputs generated prior to full model convergence can induce catastrophic numerical overflow resulting in inf generation during exponentiation. The pure function atom mitigates this hardware-level failure by explicitly bounding the t_w and t_h values (e.g., clipping at $\ln(1000/16)$) prior to exponentiation to guarantee permanent numerical stability.

Name: decode_boxes

Description: Converts raw continuous neural network delta predictions back into actionable absolute bounding box coordinates utilizing standard Faster R-CNN parameterization.

Source: https://github.com/tryolabs/object-detection-workshop/blob/master/Implementing%20Faster%20R-CNN.ipynb

License: MIT

Concept type: signal_filter

Signature: (anchors: NDArray, deltas: NDArray, scales: tuple[float, float, float, float]) -> NDArray

Pure function boundary: predefined absolute anchors and raw network floating-point deltas in; fully decoded absolute pixel bounding boxes out.

Contracts:

- require: anchors.shape == deltas.shape
- require: anchors consistently formatted as (x_min, y_min, x_max, y_max)
- ensure: output decoded widths and heights are strictly positive via defensive np.exp value clipping
  Witness: injecting a neutral delta tensor entirely populated by [0.0, 0.0, 0.0, 0.0] evaluates to return exactly the unadulterated original anchor box geometries.
  Dependencies: numpy
  CDG stages covered: rsna_series/box_decoding

Non-Standard Signal Modalities: Temporal and Mask-Based Paradigms

While classical spatial 2D bounding boxes dominate the overwhelming majority of object detection pipelines, specific computational domains generate radically different abstract coordinate formats. These instances mandate highly specialized purely functional conversions and custom signal processing heuristics to successfully interface with downstream analytics.

Temporal and Probabilistic Signal Peak Finding (1D NMS)

In advanced temporal action detection tasks—such as tracking transient soccer passes and aggressive tackles in the Kaggle DFL Bundesliga competition—the neural network predicts a continuous 1D probability signal spanning the temporal frames of a video rather than predicting standard 2D boxes.[36] The algorithmic post-processing objective is to locate the discrete, highly localized timestamps of the biological actions by isolating the peaks of this probability wave.

A naive, heavily coupled approach attempts to rely on external libraries such as scipy.signal.find_peaks.[38] While undeniably powerful, integrating scipy into an otherwise pure-NumPy deployment container introduces a massive, compiled dependency that can severely complicate highly optimized edge-device deployments. Furthermore, 1D NMS for temporal signals heavily prioritizes the specific concept of *overlap distance suppression* rather than merely executing topographic prominence filtration.[39]

A pure NumPy implementation of 1D NMS efficiently identifies local maxima and applies proximity suppression simultaneously. First, it isolates potential waveform peaks by logically comparing the probability signal strictly against its immediate neighbors: is_peak = (signal[1:-1] > signal[:-2]) & (signal[1:-1] > signal[2:]).[40]

The integer indices of these local maxima are gathered into an array and sorted sequentially by their associated probability scores. Iterating through the sorted peaks from highest confidence to lowest, the algorithm computes the absolute distance (measured in video frames or raw seconds) to all previously retained, higher-scoring peaks. If the calculated distance is decisively below a specified min_distance threshold (serving essentially as the 1D temporal equivalent of the classical IoU threshold), the lower-scoring peak is permanently suppressed.[40] This mathematical guarantee provides a highly efficient, deterministic approximation of temporal action localization constructed entirely within the rigid bounds of standard array operations.

Name: nms_1d

Description: Detects discrete temporal peaks in a 1D probability signal by successfully isolating local maxima and applying rigorous temporal distance-based non-maximum suppression.

Source: https://www.geeksforgeeks.org/data-analysis/peak-signal-detection-in-real-time-time-series-

data/

License: CC BY-SA 4.0 / Re-implementable under public domain logic

Concept type: signal_filter

Signature: (signal: NDArray, min_distance: int, threshold: float) -> NDArray

Pure function boundary: dense 1D temporal probability array in; isolated 1D array of valid peak indices out.

Contracts:

- require: signal.ndim == 1
- require: min_distance > 0
- require: 0.0 <= threshold <= 1.0
- ensure: output peak indices are permanently separated by a gap of at least min_distance
- ensure: signal[output] >= threshold is maintained for all output indices
  Witness: evaluating a signal exhibiting a broad, high-probability flat plateau systematically drops all adjacent points, retaining only a single authoritative peak representing the absolute center of the action.
  Dependencies: numpy
  CDG stages covered: dfl_bundesliga/nms_1d

Mask-to-Box Spatial Conversion and Projection

In dense panoptic or instance segmentation models (such as the widely deployed Mask R-CNN), the initial or intermediate neural network spatial output often manifests in the form of dense, pixel-perfect binary masks formatted as $$, where N represents the finite number of distinct predicted objects.[41] To interface these dense biological or mechanical predictions with standard downstream detection metrics, simplified object evaluation tools, or classical bounding-box tracking pipelines, the binary masks must be systematically converted into rectilinear bounding box coordinates defining the extreme boundaries of the segmented object.[41]

Libraries such as Torchvision provide built-in masks_to_boxes operations, but bridging this specific logic into a completely pure NumPy workflow requires a highly vectorized implementation to circumvent the catastrophic performance degradation triggered by iterating over immense 2D pixel grids using standard Python loops.[42]

The most memory-efficient pure NumPy approach, known colloquially in computer vision processing as "Downright Boxing" or mathematical boolean projection, aggressively utilizes the np.any operator. By aggressively projecting the dense 3D mask array uniformly across its internal spatial axes, the algorithm successfully collapses the heavy 2D mask into a pair of lightweight 1D boolean arrays:

- The X-axis projection evaluates: x_projection = np.any(masks, axis=1) (Yielding an optimal

shape of $$)

- The Y-axis projection evaluates: y_projection = np.any(masks, axis=2) (Yielding an optimal shape of $[N, H]$)

For each target mask N, the optimal bounding box is perfectly defined by the spatial locations of the first and last True boolean indices in these collapsed projections. Extracting np.argmax(x_projection, axis=1) instantly locates the precise x_min bound. Identifying the x_max bound requires temporarily flipping the projection memory (x_projection[:, ::-1]), invoking the argmax evaluation, and mathematically subtracting the result from the total array width. This entire boolean operation is comprehensively vectorized, operating flawlessly on dense batches containing thousands of high-resolution masks without a single pixel-level Python loop.[44] A crucial algorithmic edge case exists in handling completely empty biological masks (arrays populated entirely by False flags); the functional contract stringently enforces that these degenerate inputs evaluate to yield an invalid or empty bounding box (e.g., ``) to aggressively prevent downstream geometry errors.

Name: masks_to_boxes

Description: Efficiently converts dense binary instance masks into absolute bounding box coordinates utilizing highly vectorized boolean axis projections.

Source: https://supervision.roboflow.com/0.21.0/detection/utils/

License: MIT

Concept type: analysis

Signature: (binary_masks: NDArray) -> NDArray

Pure function boundary: dense boolean tensor shaped as in; absolute bounding box extrema arrays structured as [N, 4] out.

Contracts:

- require: binary_masks.dtype == bool
- require: binary_masks.ndim == 3
- ensure: output.shape == (N, 4)
- ensure: output mathematical coordinates are strictly bounded between the hard limits of and [0, H-1]
  Witness: passing an entirely False array generates a fallback of ; testing a massive mask with a single True pixel located at index (5, 5) perfectly yields bounding geometry of .
  Dependencies: numpy
  CDG stages covered: sartorius_cell_segmentation/mask_to_box

Identity Association and Data Thresholding

Operations

The final analytical stage of advanced deep learning post-processing architectures frequently bridges the temporal gap between isolated, static image detections and sophisticated multi-frame continuous reasoning. Robust object trackers fundamentally depend on the mathematical association of an array of newly predicted bounding boxes originating in temporal frame T with existing historical object tracks persisting from frame $T-1$.

Multi-Object Tracking Bipartite Association

The spatial association of bounding boxes across chronological frames is formulated mathematically as a dense linear sum assignment problem, overwhelmingly solved via the classical Hungarian Algorithm.[45] Given an $N \times M$ numerical cost matrix—where the matching cost is traditionally defined as $1 - IoU(box_N, track_M)$—the overarching algorithmic objective is to successfully isolate a strictly 1-to-1 matrix matching that reliably minimizes the total global cost across all concurrent assignments.[46]

While the preceding implementations within this analytical survey heavily restricted the integration of the massive scipy dependency library to deliberately avoid monolithic deployment overhead, tracking association represents a critical, unavoidable exception.[47] Attempting to implement the highly recursive $O(n^3)$ Hungarian optimization algorithm purely within standard NumPy arrays is notoriously complex, highly unoptimized, and deeply error-prone.[48] Consequently, cleanly wrapping scipy.optimize.linear_sum_assignment represents the definitive, uncontested standard in modern tracking pipelines.[47]

The pure functional execution interface abstracts away the underlying cost matrix generation dynamics. It first computes the rigorous pairwise IoU matrix utilizing the techniques established in Section 1, mathematically subtracts those values from 1.0 to construct the required cost matrix, and finally applies a rigid spatial matching threshold. If the optimized SciPy assignment proposes a match yielding a cost higher than $1 - \text{iou\_threshold}$, the proposed pairing is completely rejected. This secondary constraint enforces the principle that two totally distinct objects merely crossing paths at high speeds are not incorrectly, permanently conflated into a single tracker ID.[49]

Bipartite Association Matrix for Multi-Object Tracking

The association algorithm minimizes total assignment cost. The initial Intersection over Union (IoU) matrix is inverted into a Cost Matrix (Cost = 1 - IoU). The highlighted cells represent the optimal 1-to-1 matching determined by the Hungarian algorithm.

Data sources: Codemia, Ultralytics GitHub

Name: associate_boxes

Description: Resolves dense multi-object tracking association dynamics by executing the linear sum assignment (Hungarian) optimization over an IoU-based distance cost matrix.

Source: https://codemia.io/knowledge-hub/path/how_to_use_scipyoptimizelinear_sum_assignment_in_tensorflow_or_keras

License: BSD-3-Clause (SciPy wrapper)

Concept type: analysis

Signature: (boxes_a: NDArray, boxes_b: NDArray, iou_threshold: float) -> tuple

Pure function boundary: sets of geometry boxes extracted from frame T-1 and T in; matched indices tuples (array A, array B) alongside distinct arrays of unmatched leftover indices out.

Contracts:

- require: bounding boxes formatted uniformly as absolute physical coordinates
- ensure: return arrays consistently contain entirely distinct, non-duplicated tracker indices
- ensure: matched geometric pairs successfully exhibit an IoU strictly greater than the baseline threshold
  Witness: two identically placed geometric boxes captured in frame A and frame B are perfectly matched (evaluating to a cost of 0), while completely disjoint bounding boxes are safely relegated to the unmatched generation pool.
  Dependencies: numpy, scipy
  CDG stages covered: nfl_helmet_assignment/association

Confidence Thresholding and Domain-Specific Extraction Constraints

The computationally simplest, yet arguably the most universally deployed analytical atom in the entire inference pipeline is absolute confidence thresholding. Long before any mathematically heavy spatial heuristics like Non-Maximum Suppression or Weighted Box Fusion are executed, the raw millions of localized predictions streaming from the network must be aggressively filtered to systematically eliminate low-confidence background noise and artifact interference.

Name: threshold_detections

Description: Systematically filters bounding boxes and linked probability scores based on a static confidence baseline utilizing fast boolean indexing.

Source: Common practice

License: Public Domain

Concept type: signal_filter

Signature: (boxes: NDArray, scores: NDArray, threshold: float) -> tuple

Pure function boundary: raw dense boxes and scores in; aggressively truncated box subsets and scores out.

Contracts:

- require: boxes.shape == scores.shape
- ensure: all remaining values in the output scores array are >= the provided threshold
  Witness: injecting an array of raw probability scores [0.1, 0.9, 0.4] governed by a threshold of 0.5 successfully retains only the geometric box associated with the 0.9 score.
  Dependencies: numpy

CDG stages covered: birdclef/thresholding, dfl_bundesliga/thresholding

Furthermore, highly specialized operational domains—such as biometric deepfake face detection utilized in the intensive DFDC Kaggle competition—heavily leverage highly specific multi-task detection models like MTCNN and RetinaFace. These sophisticated models concurrently output bounding boxes intimately tied alongside dense 5-point facial landmarks (identifying the eyes, nose, and distinct mouth corners).[50] A highly specialized post-processing extraction atom isolates these bounding boxes, but crucially, it must implement custom geometric padding ratios and strict boundary image clipping algorithms.

Because a biometric bounding box generated by these models often cuts tightly across the facial center, robust post-processing mandates synthetically expanding the resulting mathematical w and h values by a specifically tuned geometric ratio (e.g., expanding by 20%) to safely encapsulate the biological chin and upper forehead boundaries, while rigorously clipping the resulting expanded coordinates securely against the absolute, immutable image boundaries (bounded tightly by 0 and image_size).[52] This mathematically verified padding algorithm ultimately guarantees that secondary downstream biometric classification models operating on the cropped facial tensor receive the maximum contextual data envelope without triggering array out-of-bounds evaluation faults.

Conclusions and Pipeline Synthesis

The rigorous architectural paradigm of defining and encapsulating deep learning object detection post-processing mechanics strictly through discrete, pure-function computational atoms provides an unprecedented leap in deployment robustness. Tying spatial and analytical heuristics directly into rigid inference frameworks inherently risks performance bottlenecks, unpredictable state manipulation, and significant hurdles when porting mathematical operations to embedded edge hardware.

By strictly restricting the geometric evaluation boundaries (such as standard IoU, GIoU, and Multi-model Ensembling), dense redundancy filters (including Greedy NMS, Soft-NMS, and advanced WBF), and complex signal decoding parameters (Faster R-CNN coordinate deltas, 1D Sequence Peak Finding) exclusively to highly vectorized, mathematically immutable NumPy routines, this implementation avoids the pervasive "deployment lock-in" typical of expansive deep learning libraries. Every function documented herein is mathematically deterministic, heavily constrained by explicit data verification contracts, and optimized comprehensively for low-level cache locality and continuous C-backend multi-threading. Embracing this pure functional design methodology directly enables practitioners to deliver best-in-class, mathematically verifiable performance stability across a vastly diverse spectrum of modern computer vision processing pipelines.

Works cited

1. Why vectorized operations in NumPy are faster? | by John Lu - Medium, accessed

April 28, 2026, https://lush93md.medium.com/why-vectorized-operations-in-numpy-are-faster-ad61d47b7016
2. How to Code Non-Maximum Suppression (NMS) in Plain NumPy - Roboflow Blog, accessed April 28, 2026, https://blog.roboflow.com/how-to-code-non-maximum-suppression-nms-in-plain-numpy/
3. How to vectorize pairwise (dis)similarity metrics - Towards Data Science, accessed April 28, 2026, https://towardsdatascience.com/how-to-vectorize-pairwise-dis-similarity-metrics-5d522715fb4e/
4. numpy python: vectorize distance function to calculate pairwise distance of 2 matrix with a dimension of (m, 3) - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/60039982/numpy-python-vectorize-distance-function-to-calculate-pairwise-distance-of-2-ma
5. np.array[:,None,:2] confusion, IoU calculation issue - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/72288880/np-array-none-2-confusion-iou-calculation-issue
6. Whats the fastest method for iou in numpy? - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/74266270/whats-the-fastest-method-for-iou-in-numpy
7. 20+ Types of Loss Functions in Machine Learning - Analytics Vidhya, accessed April 28, 2026, https://www.analyticsvidhya.com/blog/2026/04/types-of-loss-functions-in-machine-learning/
8. Loss Functions and Metrics in Deep Learning, accessed April 28, 2026, https://arxiv.org/html/2307.02694
9. Reference for ultralytics/utils/metrics.py, accessed April 28, 2026, https://docs.ultralytics.com/reference/utils/metrics/
10. iou - Raster Vision v0.31.1 documentation, accessed April 28, 2026, https://docs.rastervision.io/en/0.31/api_reference/_generated/rastervision.core.data.label.tfod_utils.np_box_ops.iou.html
11. Intuition and Implementation of Non-Max Suppression Algorithm in Object Detection, accessed April 28, 2026, https://medium.com/data-science/intuition-and-implementation-of-non-max-suppression-algorithm-in-object-detection-d68ba938b630
12. remydubois/lsnms: Large Scale Non maximum Suppression. This project implements a O(nlog(n)) approach for non max suppression, useful for object detection ran on very large images such as satellite or histology, when the number of instances to prune becomes very large. · GitHub, accessed April 28, 2026, https://github.com/remydubois/lsnms
13. What is Non-Maximum Suppression? - GeeksforGeeks, accessed April 28, 2026, https://www.geeksforgeeks.org/computer-vision/what-is-non-maximum-suppres

sion/
14. Soft-NMS -- Improving Object Detection With One Line of Code - CVF Open Access, accessed April 28, 2026, https://openaccess.thecvf.com/content_ICCV_2017/papers/Bodla_Soft-NMS_--_Improving_ICCV_2017_paper.pdf
15. [1704.04503] Soft-NMS -- Improving Object Detection With One Line of Code - arXiv, accessed April 28, 2026, https://arxiv.org/abs/1704.04503
16. soft-nms/py_nms.py at master · OneDirection9/soft-nms · GitHub, accessed April 28, 2026, https://github.com/OneDirection9/soft-nms/blob/master/py_nms.py
17. Soft-NMS-1/soft_nms.py at master - GitHub, accessed April 28, 2026, https://github.com/Lechatelia/Soft-NMS-1/blob/master/soft_nms.py
18. [1910.13302] Weighted boxes fusion: Ensembling boxes from different object detection models - arXiv, accessed April 28, 2026, https://arxiv.org/abs/1910.13302
19. Weighted Boxes Fusion — A detailed view | by Sambasivarao. K | Analytics Vidhya | Medium, accessed April 28, 2026, https://medium.com/analytics-vidhya/weighted-boxes-fusion-86fad2c6be16
20. 2nd Place - Weighted Box Fusion and Post Process - Kaggle, accessed April 28, 2026, https://www.kaggle.com/competitions/feedback-prize-2021/writeups/feedforward-2nd-place-weighted-box-fusion-and-post
21. Set of methods to ensemble boxes from different object detection models, including implementation of "Weighted boxes fusion (WBF)" method. - GitHub, accessed April 28, 2026, https://github.com/zfturbo/weighted-boxes-fusion
22. Mixture of Expert-Based SoftMax-Weighted Box Fusion for Robust Lesion Detection in Ultrasound Imaging - PMC, accessed April 28, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11899514/
23. Digging into Detectron 2 — part 4 | by Hiroto Honda - Medium, accessed April 28, 2026, https://medium.com/@hirotoschwert/digging-into-detectron-2-part-4-3d1436f91266
24. Faster R-CNN Object Detector | ArcGIS API for Python - Esri Developer, accessed April 28, 2026, https://developers.arcgis.com/python/latest/guide/faster-rcnn-object-detector/
25. detectron2.modeling, accessed April 28, 2026, https://detectron2.readthedocs.io/en/v0.6/modules/modeling.html
26. FPN.py - facebookresearch/Detectron - GitHub, accessed April 28, 2026, https://github.com/facebookresearch/Detectron/blob/master/detectron/modeling/FPN.py
27. RetinaNet Object Detection in Python | A Name Not Yet Taken AB, accessed April 28, 2026, https://www.annytab.com/retinanet-object-detection-in-python/
28. API Reference — MMDetection 2.12.0 documentation, accessed April 28, 2026, https://mmdetection.readthedocs.io/en/v2.12.0/api.html
29. Tuning first_stage_anchor_generator in faster rcnn model - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/52047638/tuning-first-stage-anchor-genera

tor-in-faster-rcnn-model
30. numpy.meshgrid — NumPy v2.2 Manual, accessed April 28, 2026, https://numpy.org/doc/2.2/reference/generated/numpy.meshgrid.html
31. Basic grid generation — pygridgen 0.2.0 documentation - GitHub Pages, accessed April 28, 2026, https://pygridgen.github.io/pygridgen/tutorial/basics.html
32. detectron2/detectron2/modeling/anchor_generator.py at main · facebookresearch/detectron2 - GitHub, accessed April 28, 2026, https://github.com/facebookresearch/detectron2/blob/main/detectron2/modeling/anchor_generator.py
33. object-detection-workshop/Implementing Faster R-CNN.ipynb at master - GitHub, accessed April 28, 2026, https://github.com/tryolabs/object-detection-workshop/blob/master/Implementing%20Faster%20R-CNN.ipynb
34. Understanding and Implementing Faster R-CNN: A Step-By-Step Guide - Medium, accessed April 28, 2026, https://medium.com/data-science/understanding-and-implementing-faster-r-cnn-a-step-by-step-guide-11acfff216b0
35. What is the purpose of the scales factor in Faster Rcnn Box Coder? - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/58026964/what-is-the-purpose-of-the-scales-factor-in-faster-rcnn-box-coder
36. DFL - Bundesliga Data Shootout Analysis using YOLO, OpenCV, and Python - GitHub, accessed April 28, 2026, https://github.com/aaditya29/DFL-Bundesliga-Data-Shootout-Analysis
37. DFL - Bundesliga Data Shootout | Kaggle, accessed April 28, 2026, https://www.kaggle.com/competitions/dfl-bundesliga-data-shootout
38. find_peaks — SciPy v1.17.0 Manual, accessed April 28, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html
39. Peak-finding algorithm for Python/SciPy - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/1713335/peak-finding-algorithm-for-python-scipy
40. Peak Signal Detection in Real-Time Time-Series Data - GeeksforGeeks, accessed April 28, 2026, https://www.geeksforgeeks.org/data-analysis/peak-signal-detection-in-real-time-time-series-data/
41. Repurposing masks into bounding boxes — Torchvision 0.11.0 documentation, accessed April 28, 2026, https://docs.pytorch.org/vision/0.11/auto_examples/plot_repurposing_annotations.html
42. masks_to_boxes — Torchvision main documentation, accessed April 28, 2026, https://docs.pytorch.org/vision/main/generated/torchvision.ops.masks_to_boxes.html
43. Computing Bounding Boxes from a Mask-Image (Tensorflow or other) - Stack Overflow, accessed April 28, 2026,

https://stackoverflow.com/questions/73282135/computing-bounding-boxes-from-a-mask-image-tensorflow-or-other
44. Utils - Supervision, accessed April 28, 2026, https://supervision.roboflow.com/0.21.0/detection/utils/
45. how to use scipy.optimize.linear_sum_assignment in ... - Codemia, accessed April 28, 2026, https://codemia.io/knowledge-hub/path/how_to_use_scipyoptimizelinear_sum_assignment_in_tensorflow_or_keras
46. Linear Sum Assignment Solver | OR-Tools - Google for Developers, accessed April 28, 2026, https://developers.google.com/optimization/assignment/linear_assignment
47. linear_sum_assignment — SciPy v1.17.0 Manual, accessed April 28, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html
48. Notebook 2.5 - The Hungarian Algorithm, accessed April 28, 2026, https://transport-systems.imperial.ac.uk/tf/60008_21/n2_5_hungarian_algorithm.html
49. Discussion of IoU based matching · Issue #5351 - GitHub, accessed April 28, 2026, https://github.com/ultralytics/ultralytics/issues/5351
50. Detection Parameters - MTCNN Documentation - Read the Docs, accessed April 28, 2026, https://mtcnn.readthedocs.io/en/latest/usage_params/
51. Creating a Face Recognition System with MTCNN, FaceNet, and Milvus | by Devraj Agarwal, accessed April 28, 2026, https://medium.com/@devraj.agarwal/creating-a-face-recognition-system-with-mtcnn-facenet-and-milvus-e155c36d8852
52. How to face extraction from images in a folder with MTCNN in python? - Stack Overflow, accessed April 28, 2026, https://stackoverflow.com/questions/65105644/how-to-face-extraction-from-images-in-a-folder-with-mtcnn-in-python
53. Py-Feat: Python Facial Expression Analysis Toolbox - PMC - NIH, accessed April 28, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10751270/